

VERSION 2.0 - ARCHITECTURE EXTRÊME & CHATBOT

ÉTUDE DE FAISABILITÉ & CAHIER DES CHARGES AVANCÉ

*Plateforme Multi-Médias NoDB avec Chatbot IA Conversational, Stealth
Scraping et Vector Search Côté Client*



Projet : Média-Recommender NoDB & Chatbot IA

Date : Juillet 2026

Concepteur : Antigravity AI Code Assistant

Stack : 100% Code Open-Source & Hébergement Gratuit (HF Spaces / Vercel / Gemini)

Sommaire Executif (V2.0)

Cette version 2.0 réévalue en profondeur l'architecture globale pour repousser les limites de la performance, de la furtivité du scraping et de l'expérience utilisateur grâce à l'intégration d'un **Chatbot IA Conversationnel** interactif avec capacité de **Function Calling**.

Chapitre	Page
1. Analyse de Faisabilité Extrême & Architecture Zero-DB	3
1.1 Stealth Scraping & Emulation TLS / HTTP2	
1.2 Pre-Scraping & Micro-Caching Vectoriel Côté Client (Transformers.js)	
1.3 Zero-Database State Management	
2. Intégration du Chatbot IA Conversationnel	5
2.1 Dialogue Multi-Tours & Intent Parsing	
2.2 LLM Function Calling pour Déclenchement de Scrapers	
2.3 Streaming Bidirectionnel (Server-Sent Events)	
3. Cadre Légal, Éthique et Limites Étroites ("Limite-Limite")	6
3.1 Scraping Passif vs Bypass d'Anti-Bot	
3.2 Conformité TDM / RGPD & Anonymisation Totale	
4. Cahier des Charges Fonctionnel Complété (CDCF)	7
4.1 Scénarios et User Stories du Chatbot	
4.2 Matrice Complète des Fonctionnalités	
5. Cahier des Charges Technique & Architecture APIs (CDCT)	8
5.1 Spécification OpenAPI / JSON Schema des Fonctions	
5.2 Protocoles d'Échange SSE & Fallbacks APIs	
6. Alternatives Techniques & Modèles Comparatifs	9
6.1 Benchmark des Scrapers & Moteurs d'IA	
6.2 Comparatif des Cloud Providers Gratuits	
7. Guide de Codage & Déploiement Extrême (100% Free)	10
7.1 Structure Complete du Code Backend & Agent IA	
7.2 Dockerfile & Orchestration Cloud Hugging Face Spaces	

1. Analyse de Faisabilité Extrême & Architecture Zero-DB

Pour offrir l'expérience utilisateur la plus rapide possible tout en conservant la contrainte stricte de 0€ de coût de serveur et 0 base de données cliente/serveur, nous poussons les techniques web à leur niveau extrême.

1.1 Stealth Scraping et Émulation TLS/HTTP2

Les scrapers traditionnels sont facilement bloqués par les pare-feux d'application web (WAF) comme Cloudflare ou Akamai. Pour pousser la collecte de données à la limite de l'indélectabilité légale :

- **Emulation de Empreinte TLS/JA3** : Au lieu d'utiliser la pile HTTP standard de Node.js ou Python qui trahit le client automatique, nous utilisons des connecteurs comme `got-scraping` (JS) ou `curl_cffi` (Python). Ces outils répliquent à l'octet près la poignée de main TLS (ciphers, extensions, HTTP/2 SETTINGS) d'un vrai navigateur Chrome ou Firefox sur Linux/Windows.
- **Stealth Puppeteer en Fallback** : Si une page nécessite l'exécution JS, nous exécutons un conteneur headless Chrome avec `puppeteer-extra-plugin-stealth` préconfiguré pour masquer la présence de `navigator.webdriver` et simuler des mouvements de souris aléatoires.
- **Proxies Edge Gratuits** : Utilisation de Cloudflare Workers gratuits (100 000 requêtes/jour) configurés comme micro-relais pour faire pivoter les adresses IP d'origine sans déboursier un centime.

1.2 Pre-Scraping & Micro-Caching Vectoriel Côté Client

Plutôt que d'interroger le serveur à chaque lettre tapée, nous repoussons l'intelligence directement dans le navigateur du client grâce aux fonctionnalités web modernes :

1. **Client-Side Embeddings avec Transformers.js** : Nous embarquons un micro-modèle d'embeddings (`all-MiniLM-L6-v2`) directement dans le navigateur via WebAssembly. Dès que l'utilisateur écrit une intention, la vectorisation s'effectue en local en moins de 15ms.
2. **Vector Cache dans IndexedDB** : Les résultats des recherches précédentes et les métadonnées scrapées sont sauvegardés dans l'IndexedDB du navigateur sous forme de vecteurs. Si l'utilisateur demande des médias similaires à une recherche effectuée la veille, la réponse est **instantanée** (0ms de latence réseau).

Technique Extrême : Edge Pre-fetching

Un script périodique pré-aspirer les listes de sitemaps XML des sites cibles (IMDb, MyAnimeList, SensCritique, BD Gest) pour extraire les dernières nouveautés et les injecter dans un fichier JSON statique mis en cache CDN sur GitHub Pages. Le client charge ce fichier de 2 Mo au démarrage : le scraping n'a

même plus besoin d'accéder aux sites distants pour 80% des médias populaires !

2. Intégration du Chatbot IA Conversationnel

L'innovation majeure de cette version 2.0 est la transformation de la simple barre de recherche en un **Chatbot IA Conversationnel Interactif**. L'utilisateur ne remplit plus un formulaire rigide : il discute naturellement avec un agent spécialisé en culture média.

2.1 Architecture du Chatbot & Function Calling

Le chatbot est propulsé par un modèle de langage (ex: **Gemini 1.5 Flash API**) configuré avec du *Function Calling* (Tool Use). Le bot dispose de trois fonctions clés qu'il peut invoquer en temps réel au cours de la discussion :

```

????????????????????????????????????????????????????????????????????????????????
?                                     INTERFACE DU CHATBOT IA                                     ?
?  User: "Je veux un anime cyberpunk sombre, mais pas Ghost in the Shell" ?
????????????????????????????????????????????????????????????????????????????????
?                                     ?
?                                     ?
????????????????????????????????????????????????????????????????????????????????
?                                     AGENT IA (GEMINI / MISTRAL)                                     ?
?  - Analyse l'intention utilisateur ?
?  - Décide d'exécuter l'outil: trigger_scraping(type="anime", query="cyberpunk")?
????????????????????????????????????????????????????????????????????????????????
?                                     ?
?                                     ?
????????????????????????????????????????????????????????????????????????????????
?                                     MOTEUR DE SCRAPING STEALTH (PARALLÈLE)                                     ?
?  - Scrape MyAnimeList / AniList en temps réel ?
?  - Renvoie 5 fiches JSON au Chatbot ?
????????????????????????????????????????????????????????????????????????????????
?                                     ?
?                                     ?
????????????????????????????????????????????????????????????????????????????????
?                                     RÉPONSE STREAMÉE AU CHATBOT (SSE)                                     ?
?  Bot: "Voici 3 animes correspondant parfaitement à tes critères..." ?
?  [Affiche les cartes interactives des animes directement dans le chat] ?
????????????????????????????????????????????????????????????????????????????????

```

2.2 Fonctions Exposées au Chatbot (Tools Schema)

- `search_media_scraping(query, media_types, filters)` : Déclenche le pipeline de scraping furtif multi-sources et retourne les fiches brutes.
- `fetch_media_details(media_url)` : Extrait en direct les avis utilisateurs et les notes détaillées d'un film ou d'une BD spécifique.
- `filter_recommendations_by_mood(mood, constraints)` : Applique un filtre de sentiment/ambiance sur les titres en cache.

3. Cadre Légal, Éthique et Limites Étroites ("Limite-Limite")

Pour opérer "à la limite" sans jamais franchir l'illégalité ni risquer d'amende CNIL ou de poursuites judiciaires, l'application respecte une charte d'ingénierie stricte.

3.1 Les Lignes Rouges à Ne Jamais Franchir

Pratique Illégale / Risquée	Pourquoi c'est Interdit	Alternative Furtive Légale Retenue
Bypass forcé de CAPTCHA (ex: 2Captcha, AntiCaptcha)	Violation de l'art. 323-1 du Code Pénal (accès frauduleux à un STAD).	Si un site présente un CAPTCHA, le scraper abandonne immédiatement et bascule sur une API alternative gratuite ou le cache sitemap.
Hébergement local des affiches et images	Violation du droit d'auteur et de la propriété intellectuelle.	Utilisation exclusive de liens chaînés direct (Hotlinking) vers les serveurs officiels ou de placeholders SVG dynamiques.
Revente ou monétisation des données brutes scrapées	Concurrence déloyale et parasitisme commercial (Directive TDM UE).	Projet 100% gratuit, sans publicité, à but personnel/démo pédagogique uniquement.
Stockage de logs comportant des IPs d'utilisateurs	Non-conformité RGPD / Sanctions CNIL jusqu'à 4% du CA.	Architecture No-DB : 0 log d'IP conservé . Tout s'efface à la fermeture de la socket.

3.2 Règle d'Or : Le "Polite Scraping"

Pour rester "indétectable" sans nuire aux serveurs cibles, notre moteur limite sa charge réseau : maximum 2 requêtes simultanées par domaine cible, et injection d'un délai d'au moins 500ms entre chaque page consultée.

4. Cahier des Charges Fonctionnel Complété (CDCF)

4.1 Fonctionnalités du Chatbot et de l'Interface

L'interface utilisateur est organisée autour d'une vue de discussion moderne, inspirée des meilleures interfaces IA (ChatGPT, Claude, Perplexity), adaptée aux médias.

Scénarios d'Interaction Utilisateur :

1. **Initiation** : L'utilisateur choisit des suggestions d'amorces (ex: ? *"Trouve-moi un film de SF méconnu"* ou ? *"Quelle BD lire après Blacksad ?"*).
2. **Refinement (Affinage)** : Après une première suggestion du chatbot, l'utilisateur peut affiner par message textuel : *"Enlève les films d'animation et garde seulement ceux qui sont sortis après 2015"*. Le chatbot ré-invoque ses filtres en local sans tout rescraper.
3. **Fiches Interactives dans le Chat** : Les médias ne sont pas simplement cités sous forme de texte brut. Le Chatbot injecte des composantes UI interactives (cartes riches avec note, badge de genre, bouton "Ajouter aux favoris local", et trailer YouTube).

4.2 Matrice Complète des Fonctionnalités V2

ID	Module	Description Avancée	Technologie
F-01	Chatbot Conversationnel	Agent conversationnel IA avec streaming SSE et mémoire de conversation temporaire.	Gemini API / Vercel AI SDK
F-02	Scraper Furtif Multi-Source	Scraping asynchrone avec empreinte TLS custom et rotation d'en-têtes HTTP/2.	got-scraping / Cheerio
F-03	Vector Cache Navigateur	Indexation sémantique locale des requêtes et résultats dans l'IndexedDB client.	Transformers.js (Wasm)
F-04	Cartes Médias Interactives	Rendu dynamique de composants HTML de cartes média enrichies dans le flux du chat.	Vanilla JS / Glassmorphism
F-05	Export / Synchro Local Storage	Sauvegarde et exportation JSON de ses favoris et listes de lecture No-DB.	LocalStorage API

5. Cahier des Charges Technique & Architecture APIs

5.1 Schéma des APIs Interne (Express / FastAPI)

Route 1 : POST /api/chat (Streaming SSE)

Gère la session de conversation avec le Chatbot et transmet la réponse en flux continu (Server-Sent Events).

```
// Payload de requête
{
  "messages": [
    { "role": "user", "content": "Je veux une BD d'aventure historique" }
  ],
  "client_cache_keys": ["vector_hash_123"]
}

// Réponse Streamée (Server-Sent Events)
event: tool_call
data: { "tool": "search_media_scraping", "status": "scraping_bdgest..." }

event: message_chunk
data: { "text": "J'ai trouvé 3 excellentes BD d'aventure historique pour vous :"}

event: media_card
data: { "title": "Les Passagers du Vent", "rating": "8.8", "url": "https://..." }
```

5.2 Pipeline d'Exécution du Chatbot (Code Node.js Conceptuel)

```
import { Express } from 'express';
import { GoogleGenerativeAI } from '@google/generative-ai';
import { scrapeBDGest, scrapeIMDb, scrapeMyAnimeList } from './scrapers.js';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

export async function handleChatStream(req, res) {
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');

  const model = genAI.getGenerativeModel({
    model: 'gemini-1.5-flash',
    tools: [{ functionDeclarations: [scrapingToolDeclaration] }]
  });

  const chat = model.startChat({ history: req.body.history });
  const result = await chat.sendMessageStream(req.body.message);

  for await (const chunk of result.stream) {
    if (chunk.functionCalls()) {
      // Déclencher le scraping asynchrone furtif !
      const scrapedData = await executeScrapingTools(chunk.functionCalls());
      // Renvoyer les résultats au modèle pour finaliser la réponse
    }
  }
}
```



```
const followUp = await chat.sendMessageStream([ { functionResponse: scrapedData } ]);
for await (const fuChunk of followUp.stream) {
  res.write(`data: ${JSON.stringify({ text: fuChunk.text() })}\n\n`);
}
} else {
  res.write(`data: ${JSON.stringify({ text: chunk.text() })}\n\n`);
}
}
}
```

6. Alternatives Techniques & Modèles Comparatifs

Afin de faire les choix d'ingénierie les plus pertinents et robustes, voici une analyse comparative approfondie des alternatives disponibles.

6.1 Comparatif des Moteurs de Scraping

Technologie	Vitesse	Furtivité Anti-Bot	Consommation RAM	Verdict
Axios + Cheerio	? Ultra-rapide (< 200ms)	?? Faible (Pas d'éval JS)	? Faible (~20 Mo)	Recommandé pour 80% des cibles HTML simples.
got-scraping	? Très rapide (< 400ms)	? Forte (Émulation TLS/HTTP2)	? Faible (~30 Mo)	Choix Principal pour contourner Cloudflare basique.
Puppeteer Stealth	? Lente (1.5s - 4s)	? Maximale (Évalue le JS)	? Élevée (~300 Mo / instance)	Fallback uniquement si blocage dur.

6.2 Comparatif des Modèles de Langage (LLM Gratuits)

Modèle LLM	Quota Gratuit	Vitesse d'inférence	Support Function Calling
Google Gemini 1.5 Flash	? 15 Requêtes / Min (Gratuit)	? Ultra-rapide (~300ms)	? Excellent (Natif)
Hugging Face Serverless (Mistral-7B)	? Illimité (Rate limit souple)	? Moyenne (~1.2s)	?? Nécessite du prompt parsing
Groq API (Llama 3 8B)	? 30 Requêtes / Min (Gratuit)	? Instantanée (< 100ms)	? Excellent (Natif)

Recommandation d'Architecture IA Duale

Utiliser **Groq API (Llama 3)** ou **Gemini 1.5 Flash** comme moteur LLM principal pour le Chatbot en raison de leur vitesse d'inférence fulgurante et du support natif du Function Calling.

7. Guide de Codage & Déploiement Extrême (100% Free)

7.1 Arborescence Finale Recommandée

```
media-recommender-v2/
??? Dockerfile           # Build conteneur pour Hugging Face Spaces
??? package.json
??? index.js             # Serveur Express & routes SSE /api/chat
??? public/              # Frontend HTML5 / CSS Glassmorphism / JS
?   ??? index.html       # Interface Chatbot & Cartes médias
?   ??? style.css        # Style Dark Mode Premium & animations
?   ??? app.js           # Client SSE, Transformers.js vector cache
?   ??? sw.js            # Service Worker (Cache offline No-DB)
??? src/
    ??? ai/
    ?   ??? chatbotAgent.js # Agent conversationnel Gemini/Groq
    ?   ??? tools.js       # Déclarations des outils Function Calling
    ??? scrapers/
        ??? stealthClient.js # Client HTTP got-scraping préconfiguré
        ??? imdbScraper.js  # Extracteur IMDb / SensCritique
        ??? malScraper.js   # Extracteur MyAnimeList / AniList
        ??? bdgestScraper.js # Extracteur BD Gest
```

7.2 Configuration Dockerfile Gratuite pour Hugging Face Spaces

```
FROM node:20-slim

# Installer Chromium pour Puppeteer Stealth (Fallback)
RUN apt-get update && apt-get install -y \
    chromium \
    fonts-liberation \
    --no-install-recommends \
    && rm -rf /var/lib/apt/lists/*

ENV PUPPETEER_SKIP_CHROMIUM_DOWNLOAD=true \
    PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium

WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .

EXPOSE 7860
CMD ["node", "index.js"]
```

Conclusion de l'Étude V2.0

Cette architecture V2 repousse les limites de ce qui est possible sans aucun budget : un Chatbot IA ultra-réactif, du scraping furtif indétectable, un cache vectoriel côté client et un déploiement cloud 100% gratuit. Le projet est prêt pour le développement !

